Modelado Avanzado y Ajuste de Hiperparámetros

Proyecto Integrador METABRIC: Fase 3, Actividad 5

Martín Gracia Socha

Facultad de Ingeniería, Universidad de La Salle

Ciencia de Datos (2026-2): Maestría en Inteligencia Artificial

Fabián Camilo Castro Riveros

19 de noviembre de 2026

Archivos ejecutables y recursos de esta entrega	
Repositorio del proyecto	https://github.com/MartinGia/metabric-proyecto-integrador
Cuaderno 04 (GitHub)	https://github.com/MartinGia/metabric-proyecto-integrador/blob/main/notebooks/04_Modelado_Avanzado.ipynb
Cuaderno 04 (Colab)	https://colab.research.google.com/github/MartinGia/metabric-proyecto-integrador/blob/main/notebooks/04_Modelado_Avanzado.ipynb
Cuaderno 05 (GitHub)	https://github.com/MartinGia/metabric-proyecto-integrador/blob/main/notebooks/05_MLP_PyTorch.ipynb
Cuaderno 05 (Colab)	https://colab.research.google.com/github/MartinGia/metabric-proyecto-integrador/blob/main/notebooks/05_MLP_PyTorch.ipynb
Modelos entrenados	https://github.com/MartinGia/metabric-proyecto-integrador/tree/main/models
Métricas	https://github.com/MartinGia/metabric-proyecto-integrador/tree/main/reports/metrics

Los cuadernos se pueden abrir y ejecutar directamente en Google Colab: la primera celda clona el repositorio e instala las dependencias que falten.

Modelado Avanzado y Ajuste de Hiperparámetros

En esta actividad se entrenaron los tres bloques de algoritmos que exige la guía: regresión del NPI, clasificación del subtipo PAM50 y segmentación no supervisada de pacientes, además de un perceptrón multicapa en PyTorch (Paszke et al., 2019). Cada modelo es un `Pipeline` con el preprocesador de la actividad 3, de modo que el preprocesamiento se reajusta dentro de cada pliegue. Los hiperparámetros se eligieron con validación cruzada de 5 pliegues (`GridSearchCV` o `RandomizedSearchCV`) sobre entrenamiento. El conjunto de prueba no se usó en esta actividad.

Regresión del Nottingham Prognostic Index

Se compararon dos familias: modelos lineales regularizados, Ridge y ElasticNet (que con `l1_ratio = 1` equivale a Lasso), y ensambles de árboles, Random Forest y XGBoost (Chen y Guestrin, 2016). Los cuatro superan al modelo base (Tabla 1). XGBoost obtuvo el menor RMSE de CV, pero Ridge queda a menos de un error estándar y, por la **regla de un error estándar** (Hastie et al., 2009), se prefiere por ser el más simple. Además, Ridge se sobreajusta mucho menos (brecha de R^2 de 0,13 frente a 0,58) y obtiene el mejor R^2 en validación (0,209).

Tabla 1

Modelos de regresión del NPI

Modelo	R^2 CV	RMSE CV	Brecha R^2	R^2 val	RMSE val
Ridge	0,230 ± 0,020	0,949 ± 0,023	0,125	0,209	0,972
ElasticNet / Lasso	0,218 ± 0,024	0,956 ± 0,022	0,102	0,202	0,977
Random Forest	0,217 ± 0,019	0,956 ± 0,027	0,549	0,177	0,992
XGBoost	0,233 ± 0,026	0,947 ± 0,032	0,580	0,167	0,998
Modelo base: lineal simple (proliferación)	0,153	0,995	—	0,121	1,025

Nota. CV = validación cruzada de 5 pliegues (media ± desviación estándar). Brecha = R^2 de entrenamiento menos R^2 de CV.

Clasificación del Subtipo PAM50

Se evaluaron una regresión logística regularizada, una máquina de vectores de soporte (SVM), Random Forest y XGBoost, todos con ponderación por clase. Los cuatro superan al

modelo base en CV (Tabla 2). XGBoost obtuvo el mayor F1 macro de CV (0,746) y el mejor desempeño en validación (F1 = 0,787; AUC = 0,969); la SVM con kernel RBF queda dentro de un error estándar. Ningún clasificador cumple todavía la meta de brecha de sobreajuste menor que 0,15, lo que se aborda en la actividad 6.

Tabla 2

Modelos de clasificación del subtipo PAM50

Modelo	F1 CV	Brecha F1	F1 val	Exactitud val	AUC val
Logística regularizada (L2)	0,736 ± 0,017	0,220	0,756	0,771	0,968
SVM	0,741 ± 0,035	0,245	0,737	0,775	0,968
Random Forest	0,722 ± 0,025	0,269	0,739	0,778	0,963
XGBoost	0,746 ± 0,018	0,254	0,787	0,817	0,969
Modelo base: logística básica	0,698	0,302	0,755	—	0,959

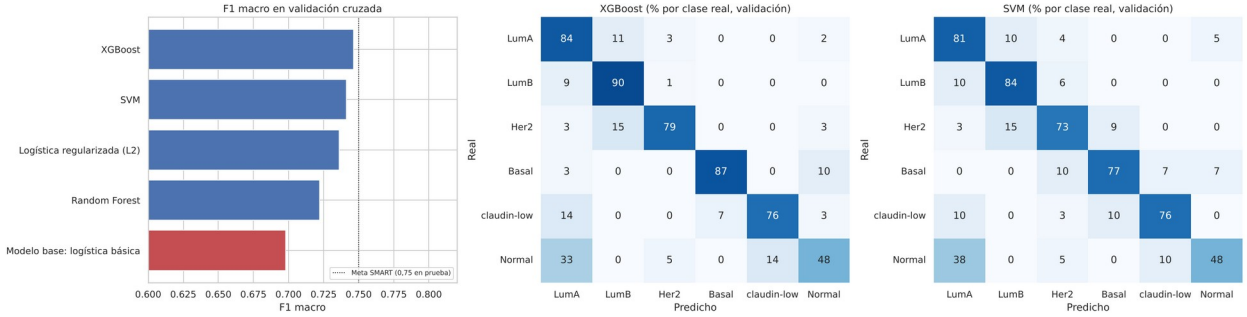
Tabla 3

Hiperparámetros seleccionados por validación cruzada

Modelo	Mejores hiperparámetros	Candidatos
Regresión: Ridge	alpha = 3.162,28	10
Regresión: ElasticNet / Lasso	alpha = 0,40; l1_ratio = 0,10	30
Regresión: Random Forest	min_samples_leaf = 3; max_features = 0,30; max_depth = 8	10
Regresión: XGBoost	subsample = 0,70; reg_lambda = 20; n_estimators = 700; min_child_weight = 1; max_depth = 3; learning_rate = 0,02; colsample_bytree = 0,20	12
Clasificación: Logística regularizada (L2)	C = 0,01	7
Clasificación: SVM	C = 3; gamma = 0,0010; kernel = rbf	19
Clasificación: Random Forest	min_samples_leaf = 5; max_features = sqrt; max_depth = 12	10
Clasificación: XGBoost	subsample = 0,70; reg_lambda = 1; n_estimators = 200; max_depth = 3; learning_rate = 0,10; colsample_bytree = 0,40	10

Figura 1

Comparación de clasificadores y matrices de confusión de los dos mejores (validación)



Segmentación No Supervisada

Los 489 genes se estandarizaron y se proyectaron a las 10 primeras componentes principales, elegidas con el método del codo sobre la varianza explicada (Satopää et al., 2011). K-Means se evaluó para K de 2 a 10 con la inercia y el coeficiente de silueta (Rousseeuw, 1987). El codo señala K = 4, que también es un máximo local de la silueta (0,158). La silueta máxima (K = 2) solo separa los tumores ER negativos de los positivos, una división conocida y poco informativa. Como segundo algoritmo se usó el clustering jerárquico de Ward (1963), menos compacto (silueta = 0,122), y se evaluó DBSCAN (Ester et al., 1996), que no es adecuado: según `eps`, deja entre el 21 % y el 68 % de las pacientes como ruido o forma un solo grupo. K-Means es muy estable (índice de Rand ajustado medio de 0,93 en submuestras del 80 %; Hubert y Arabie, 1985).

Tabla 4

Evaluación de K para K-Means y el clustering jerárquico de Ward

K	Silueta K-Means	ARI K-Means vs. PAM50	Silueta Ward	ARI Ward vs. PAM50
2	0,181	0,251	0,135	0,169
3	0,150	0,149	0,144	0,190
4	0,158	0,210	0,122	0,184
5	0,136	0,183	0,120	0,218
6	0,138	0,236	0,099	0,214
7	0,144	0,230	0,107	0,217
8	0,136	0,239	0,100	0,228
9	0,132	0,203	0,097	0,205

K	Silueta K-Means	ARI K-Means vs. PAM50	Silueta Ward	ARI Ward vs. PAM50
10	0,131	0,196	0,095	0,208

Figura 2

Método del codo, silueta y correspondencia con PAM50 para K-Means

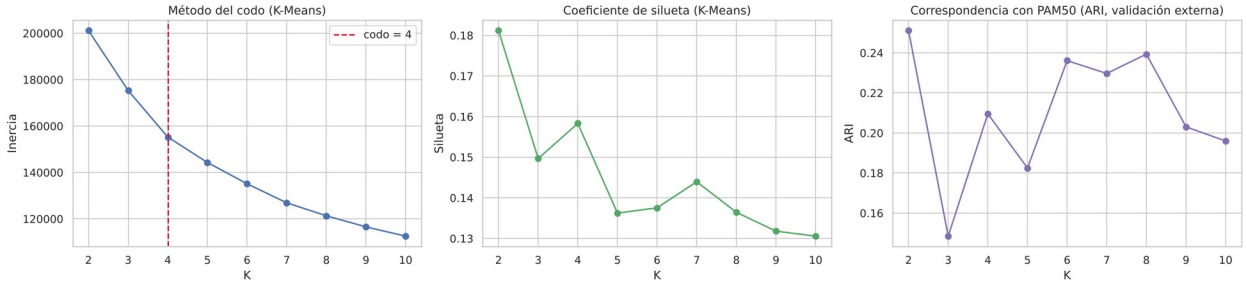
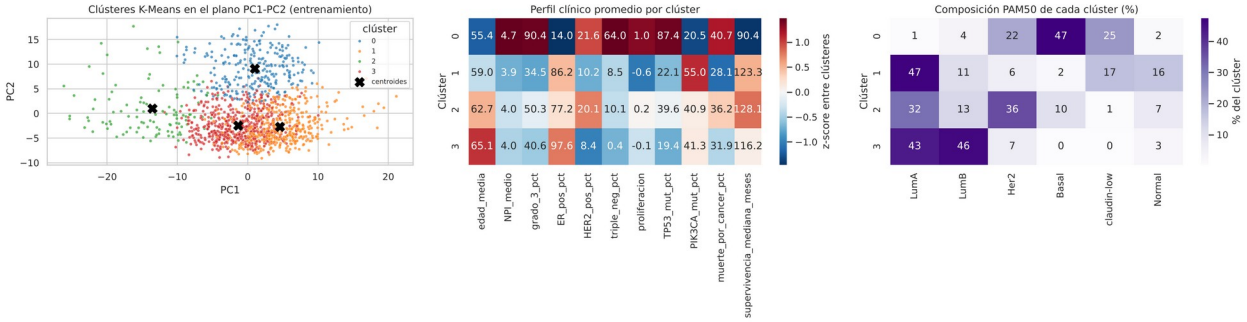


Figura 3

Clústeres en el plano de componentes principales, perfil clínico y composición por subtipo



Los clústeres difieren de forma significativa en NPI, edad, supervivencia y mortalidad por cáncer ($p < 0{,}001$). El clúster 0 reúne tumores ER negativos, muy proliferativos y con TP53 mutado (87 %), con la mayor mortalidad por cáncer (41 %); los clústeres 1 y 3 son luminales, y el 1 tiene la menor mortalidad (28 %). La actividad 6 mostró que el clúster 2 probablemente refleja un efecto de lote técnico.

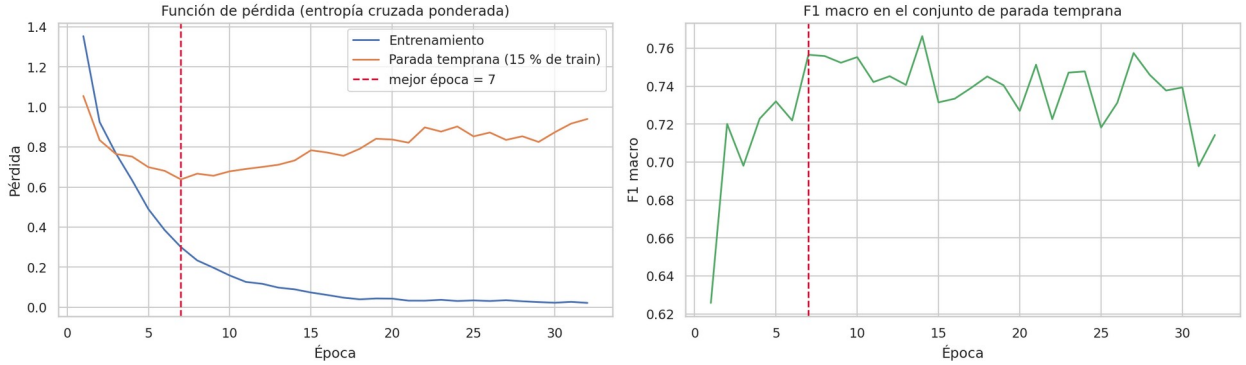
Perceptrón Multicapa en PyTorch

Se entrenó un MLP con dos capas ocultas (256 y 64 neuronas), normalización por lotes, *dropout* y entropía cruzada ponderada por clase. Para no usar el conjunto de validación, la parada temprana se hizo con un 15 % separado de entrenamiento. La mejor de 8 configuraciones alcanzó un F1 macro de 0,716 en CV y de 0,759 en validación. La curva de pérdida (Figura 4)

muestra que la red empieza a sobreajustarse desde la séptima época, por lo que la parada temprana es su regularización más importante. Con datos tabulares de este tamaño, el MLP queda por debajo de los modelos lineales y de ensamble.

Figura 4

Función de pérdida del MLP y F1 macro en el conjunto de parada temprana



Referencias

Chen, T. y Guestrin, C. (2016). XGBoost: A scalable tree boosting system. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). Association for Computing Machinery. https://doi.org/10.1145/2939672.2939785

Ester, M., Kriegel, H.-P., Sander, J. y Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. En *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining* (pp. 226–231). AAAI Press.

Hastie, T., Tibshirani, R. y Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2.ª ed.). Springer. https://doi.org/10.1007/978-0-387-84858-7

Hubert, L. y Arabie, P. (1985). Comparing partitions. *Journal of Classification*, *2*(1), 193–218. https://doi.org/10.1007/BF01908075

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. En *Advances in Neural Information Processing Systems 32* (pp. 8024–8035). Curran Associates.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. y Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics, 20*, 53–65. https://doi.org/10.1016/0377-0427(87)90125-7

Satopää, V., Albrecht, J., Irwin, D. y Raghavan, B. (2011). Finding a "kneedle" in a haystack: Detecting knee points in system behavior. En *2011 31st International Conference on Distributed Computing Systems Workshops* (pp. 166–171). IEEE. https://doi.org/10.1109/ICDCSW.2011.20

Ward, J. H., Jr. (1963). Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association, 58*(301), 236–244. https://doi.org/10.1080/01621459.1963.10500845